Experiment No:5

Student Name: Sumanyu Singh
UID:23BCS10801
Branch: BE CSE
Section/Group:KRG-3B
Semester: 6th
Date of Performance:18/02/2026
Subject Name: System Design
SubjectCode:23CSH-314

Aim: To design and develop a Messenger Application that allows users to send and receive messages in real time, manage contacts, and maintain communication between users using appropriate system design and software engineering principles(Similar to WhatsApp or Facebook Messenger) .

Objective:

- To understand the **design and working of a messaging application system**.
- To implement features that allow **users to send and receive messages**.
- To design a system that manages **user accounts and contact lists**.
- To study how **real-time communication** works between multiple users.
- To apply **software design principles and system architecture** in building a messenger application.

Tools and Technologies Used:

- **Draw.io (diagrams.net)** – for designing the **system architecture and diagrams**.
- **Programming Language:** JavaScript / Python (for application development).
- **Frontend Technologies:** HTML, CSS, React (for user interface).
- **Backend Technologies:** Node.js / Flask (for server-side logic and API handling).
- **Database:** MongoDB / MySQL (for storing user data and messages).
- **Version Control:** Git and GitHub for source code management.

Functional Requirements :

• Users must be able to **register accounts and securely log into the messenger platform**.
• The system should allow users to **send and receive text messages in real time**.
• Users should be able to **add, remove, and manage contacts or friends**.
• The application must allow users to **view chat history and previous conversations**.
• Users should be able to **receive notifications when a new message arrives**.
• The system should allow users to **create group chats and communicate with multiple users simultaneously**.
• Users may be able to **share media such as images, files, or emojis within chats**.

Non - Functional Requirements :

• **Scalability**
The messenger platform should efficiently support a large number of users and simultaneous conversations by using scalable servers and distributed system architecture.

• **Availability and Reliability**
The system should remain available most of the time so users can send and receive messages without interruption, even during high traffic or minor server failures.

• **Latency**
Message delivery time should be very low (ideally within a few milliseconds) to ensure near **real-time communication** between users.

• **Security and Privacy**
User data and messages must be protected using authentication, secure connections (HTTPS), and encryption to prevent unauthorized access.

• **Usability**
The application should have a **simple, responsive, and user-friendly interface** so users can easily navigate chats, contacts, and settings across different devices.

Core-Entites of the System:

- User
- Groups
- Messages

API Designing (Low Level Design – LLD)

1. POST API – User Registration
https://localhost/users/register
• Allows new users to create an account on the messenger platform.
• Stores user details such as username, email, and password in the database.
• Validates input data before creating the account.

2. POST API – User Login
https://localhost/users/login
• Authenticates registered users.
• Generates authentication tokens (JWT/session token) for secure access.

3. GET API – Get User Profile
https://localhost/users/{user_id}
• Retrieves profile information of a specific user.
• Returns details such as username, status, and profile picture.

4. POST API – Send Message
https://localhost/messages/send
• Allows a user to send a message to another user or group.
• Stores the message content, sender ID, receiver ID, and timestamp in the database.

5. GET API – Fetch Chat Messages
https://localhost/messages/{chat_id}

• Retrieves the message history of a particular chat or conversation.
• Returns messages in chronological order.

6. POST API – Create Group Chat
https://localhost/groups/create
• Allows users to create a group chat with multiple participants.
• Stores group name, creator ID, and member list.

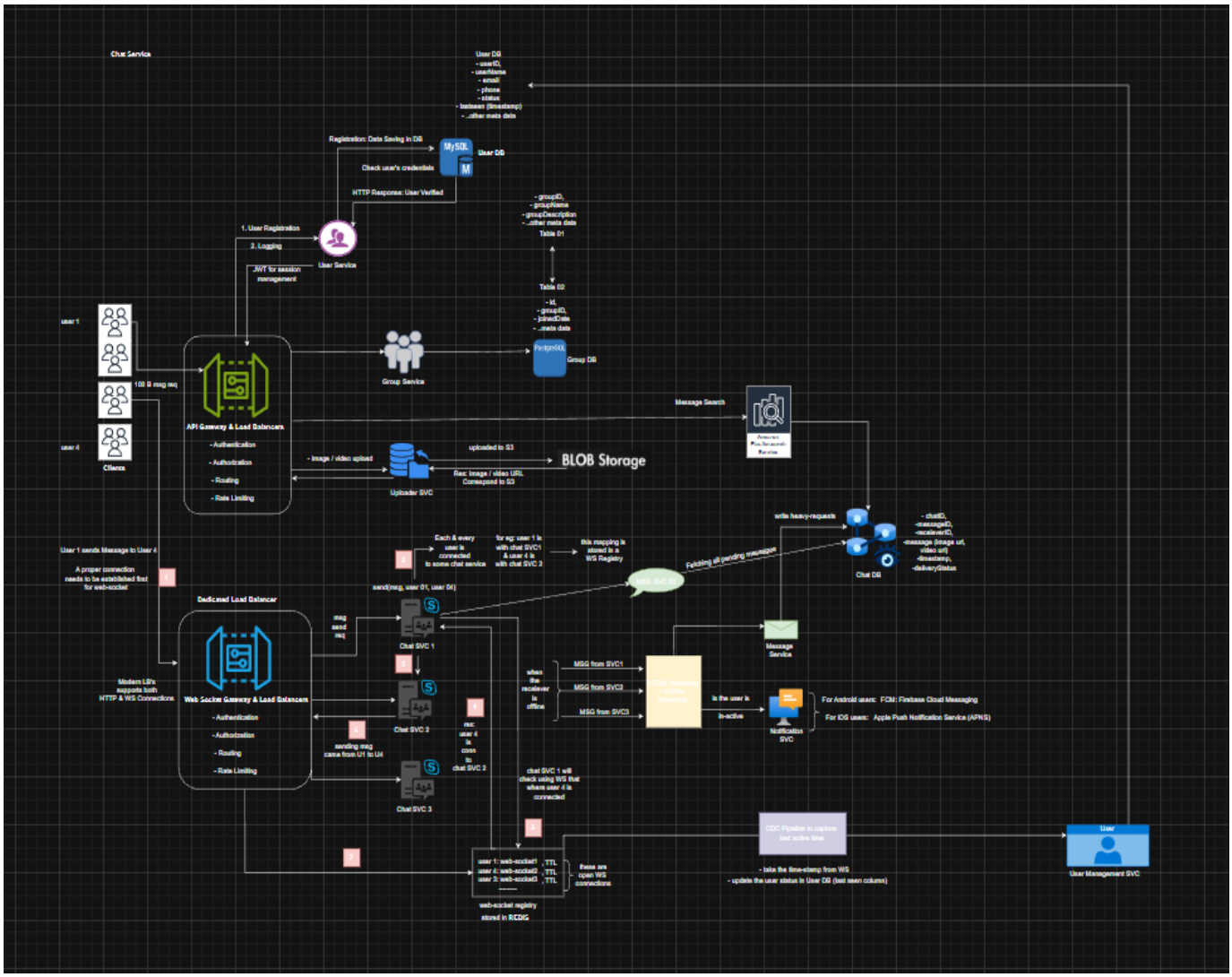
7. POST API – Add User to Group
https://localhost/groups/{group_id}/add_user
• Adds a new member to an existing group chat.
• Updates the group member list in the database.

8. GET API – Get Contact List
https://localhost/contacts/{user_id}
• Retrieves the contact or friend list of a user.
• Displays available users for starting conversations.

9. POST API – Message Delivery Status Update
https://localhost/messages/status
• Updates the message status such as sent, delivered, or read.
• Helps maintain real-time chat indicators.

10. GET API – Notifications
https://localhost/notifications/{user_id}
• Fetches notifications for new messages or group activities.
• Alerts users about unread messages or updates.

HLD(High level Diagram) -> FINAL :



Learning outcomes:

- Understand the **system design and architecture** of a messenger application.
- Learn how to identify and define **functional and non-functional requirements** of a software system.
- Gain knowledge of **API design and low-level system design (LLD)** for communication services.
- Understand how **real-time messaging systems** handle user interactions and data exchange.
- Develop skills in **software design, documentation, and system modeling** for modern applications.